EBKit DevOps implementation and deployment

Author: Rakesh Ku Patra
Task date: 17 October 2026 in Asia Kolkata
Repository: https://github.com/Rakesh-Patra/ebready
Branch: `feat/modern-workspace-deployment-improvements`

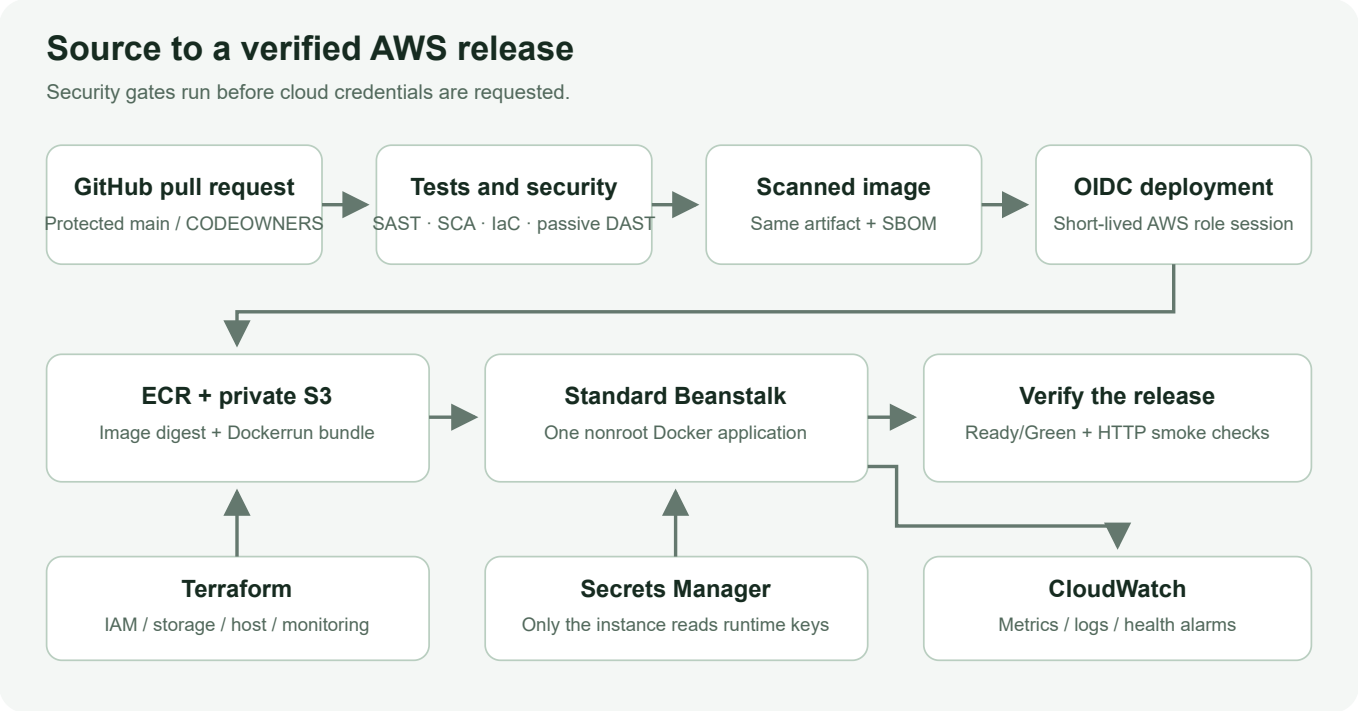
This guide explains how an application idea becomes a tested Docker image and a repeatable AWS deployment. The practical project is EBKit, a React dashboard with a Node API and Python deployment tools. The target for this exercise is the dashboard itself on a **standard Elastic Beanstalk Docker environment**. That is separate from the AWS Cluster Mode environments that EBKit can manage for users.

The aim is to demonstrate decisions, automation, security, validation and recovery. Using more tools is not the goal. Ansible is unnecessary because Beanstalk manages the host. A separate Prometheus or Grafana deployment is unnecessary for this demo; CloudWatch dashboard, enhanced Beanstalk health, CPU and health alarms, and seven days of application/platform logs provide the requested baseline.

What each tool does

- **Git and GitHub** preserve source history and isolate work on a feature branch.
- **CI** installs dependencies, runs tests, builds the UI and container, and checks infrastructure syntax.
- **DevSecOps checks** scan application dependencies, source secrets and the container before deployment credentials are requested.
- **Docker** packages the frontend, API and Python tools into one reproducible runtime. The final container runs as a nonroot user on port 8080.
- **Terraform** owns the environment, IAM roles, release bucket and image repository. A reviewed plan is applied before the first delivery.
- **OIDC** exchanges a GitHub identity token for a short lived AWS role session. There is no stored AWS access key in GitHub.
- **ECR** stores the already scanned image. Immutable tags prevent overwrites; the release uses the image digest.
- **S3** holds a small private versioned release ZIP containing `Dockerrun.aws.json`.
- **Elastic Beanstalk** provisions the managed Docker host, pulls the image, and reports deployment health.

Architecture



Implementation architecture diagram

Prerequisites

1. A GitHub repository and permission to configure its Actions variables and deployment environments.
2. An AWS account and a local bootstrap identity permitted to create the named Beanstalk resources, IAM roles, S3 bucket and ECR repository.
3. Terraform 1.14.9, Node 22, Python 3.12 or later, Git and the AWS CLI.
4. An existing GitHub OIDC provider at `https://token.actions.githubusercontent.com` in AWS IAM. This account already has one; Terraform reads and reuses it.
5. A reviewed current Amazon Linux 2023 Docker platform in the target region.
6. A custom domain and ACM certificate for an authenticated public deployment. The default Beanstalk hostname is not automatically HTTPS.

The initial bootstrap uses the operator's AWS profile. Subsequent application delivery uses OIDC from GitHub. Do not describe the bootstrap as an OIDC login.

Step 1 Start with a clear application boundary

The app serves the React build, `/api` and the `/insforge` authentication proxy from one origin. Server-only administration keys must never be included in the React bundle. The public InsForge URL and anonymous key are public application settings; they do not grant server administration rights.

For this practice environment, `EBKIT_PUBLIC_DEMO=true` disables the authentication proxy and removes credential entry from the UI. `EBKIT_LIVE_ENABLED=false` and `EBKIT_GITHUB_AUTO_DEPLOY=false` prevent this public dashboard from launching customer AWS jobs. Visitors can still explore the guest simulation. This is a demonstration deployment, not the full production multi-user control plane.

Step 2 Use a feature branch

```
git switch -c feat/modern-workspace-deployment-improvements
git fetch origin
git rebase origin/main
git diff --check
```

The branch was created, synced with the newer `main`, and pushed. A conflict in the infrastructure tools UI was resolved while preserving exact confirmation and Ready-state checks. A source scan found a deliberately fake AWS key in a redaction test rather than a real credential. Do not commit `.env`, AWS profiles, job state, Terraform plans or Terraform state.

Step 3 Test and build locally

From the repository root:

```
python -m pytest tests/test_aws_tls.py tests/test_managed_database.py tests/test_cluster_health_poll.py
tests/test_controls.py tests/test_production_ops.py -q
cd web
$env:EBKIT_TEST_PYTHON=(Get-Command python.exe).Source
npm.cmd ci
npm.cmd test -- --pool=threads --maxWorkers=1 --testTimeout=15000
npm.cmd run build
cd ..
```

Earlier verified baseline: 52 relevant Python tests, 223 web tests and the production build passed. New delivery and public-demo changes require their own fresh CI run; use the current Actions run as the release evidence rather than reusing baseline counts.

Windows test troubleshooting: the default test process pool timed out on this machine. A single threads worker completed. The ZIP tests also need a real Python executable through `EBKIT_TEST_PYTHON`. These are runner configuration details, not evidence that unexecuted tests passed.

Step 4 Package the application with Docker

Read the root `Dockerfile` and `.dockerignore`.

The build stage runs `npm ci` and the Vite production build. The runtime stage installs production Node dependencies and the Python CLI. It copies the built frontend and server, runs as the `node` user, exposes port 8080, and includes a health check against `/api/health`.

```
docker build -t ebkit:ci .
docker run --rm -p 8080:8080 -e EBKIT_PUBLIC_DEMO=true ebkit:ci
curl --fail http://127.0.0.1:8080/api/health
```

The local Docker daemon was unavailable during initial preparation. The GitHub runner builds and scans the actual delivery image; do not claim a local image test unless you have recorded it separately.

Step 5 Define infrastructure in Terraform

Configuration lives under `infra/terraform`. The stack creates a single-instance practice environment, CloudWatch dashboard and alarms, private encrypted release bucket, immutable ECR repository, instance profile, Beanstalk health role and GitHub deployment role. The existing GitHub OIDC identity provider is read rather than recreated.

```
$env:AWS_PROFILE='default'
terraform '-chdir=infra/terraform' init
terraform '-chdir=infra/terraform' fmt -check
terraform '-chdir=infra/terraform' validate
terraform '-chdir=infra/terraform' plan '-var-file=deployment.tfvars.json' '-out=deployment.tfplan'
terraform '-chdir=infra/terraform' apply deployment.tfplan
```

The initial reviewed plan had 16 creations and no changes or deletions to existing resources. Keep the lock file under version control. The local state and input files are ignored. State was migrated to the private encrypted, versioned S3 release bucket under `terraform/dashboard.tfstate`, with native S3 state locking. The GitHub application delivery role can access `releases/*`, not the state prefix. Do not pass secret values into Terraform variables.

A real troubleshooting event occurred during bootstrap: the newest AWS provider used an S3 Control endpoint that failed DNS resolution. Some resources had already been created. The recovery was to inspect saved state, pin AWS provider 6.26.0, verify and retain the newly created empty bucket, review a

plan with no deletions, and resume. The resumed apply created the remaining seven resources successfully. Never blindly recreate resources after a partial apply.

Step 6 Set up GitHub OIDC securely

The trust policy requires both:

```
aud = sts.amazonaws.com
sub = repo:Rakesh-Patra@147680316/ebready@1407395996:environment:production
```

GitHub's OIDC customization API reported immutable subjects for this repository. The exact `sub_claim_prefix` includes owner and repository IDs. An initial trust policy using only repository names was rejected; it was corrected to the exact immutable prefix rather than weakening the policy with a wildcard. Other repositories must query their own prefix instead of copying these IDs.

The production GitHub environment restricts deployment branches to the feature branch used for today's demonstration and `main`. The workflow additionally requires `github.ref_name == vars.DEPLOYMENT_BRANCH`. Fork pull requests run CI without a cloud identity and cannot execute the deployment job.

Only the deploy job receives `id-token: write`. It exchanges the token using `aws-actions/configure-aws-credentials`. Its role can push to this ECR repository, write the release prefix in this S3 bucket, create versions for this application, and update this one environment. Beanstalk also requires configuration permissions on its exact account-and-region platform bucket even after bootstrap; these are scoped to that bucket. Runtime object operations are limited to this environment's storage prefix (including runtime and application-source files), plus this application's shared `_embedded_extensions` prefix. Stack reads and updates are limited to its backing `awseb-ENVIRONMENT_ID-stack` CloudFormation stack. Unrelated buckets, environment runtime objects and stacks are outside these grants. Only the selected EC2/Auto Scaling inventory APIs and ECR token exchange use `Resource: *` where per-resource scoping is unavailable. Mutating stack and runtime permissions remain environment-scoped. The role cannot create IAM roles, read the Gemini secret or Terraform state, or provision the complete Terraform stack.

Repository Actions variables contain only configuration:

```
AWS_REGION
AWS_ROLE_ARN
EB_ARTIFACT_BUCKET
ECR_REPOSITORY
EB_APPLICATION
EB_ENVIRONMENT
DEPLOYMENT_BRANCH
```

`scripts/github_delivery.py configure` sets these variables and the branch policy using an existing Git Credential Manager credential in memory. It also reads the exact OIDC subject prefix into the ignored local Terraform input; apply that trust update before running delivery. It never writes or prints that credential. Do not paste a GitHub token into the script or this guide.

Step 7 Keep secrets in the right place

- AWS delivery credentials: generated temporarily through OIDC, not GitHub secrets.
- Public app settings: normal environment variables; no secrets are required for guest demo mode.
- Server secrets: the existing Gemini and InsForge administration keys are stored in AWS Secrets Manager; only their ARNs are passed through `secret_arns`. No secret values are committed or placed in Terraform state.
- Instance permission: `secretsmanager:GetSecretValue` is limited to those exact ARNs. Customer-managed KMS keys also require a reviewed `kms:Decrypt` grant.
- Future database passwords and connection URLs: use Secrets Manager references, scoped to the backend or workload that needs them.
- Future webhook signing secrets or private integration tokens: store them in Secrets Manager and verify signatures; do not expose them through the UI.
- TLS private keys: use ACM-managed certificates rather than committing certificate/key files.
- GitHub authentication for operator setup: stays in Git Credential Manager on the operator machine, not in the image or the repository.
- React settings: anything named `VITE_*` is browser-visible. Never place passwords, admin keys or cloud credentials there.
- Artifacts: the release ZIP contains only `Dockerrun.aws.json`. The scanned Docker image contains application source and runtime dependencies, not local environment files.

Beanstalk's `aws:elasticbeanstalk:application:environmentsecrets` namespace resolves the secret during instance bootstrap on supported platform versions. Changing a secret does not automatically prove that a running app has refreshed it; perform and verify the appropriate environment refresh/redeployment.

Step 8 Run CI and security gates

The pinned workflow is `.github/workflows/delivery.yml`.

The verification job runs relevant Python tests, all web tests, frontend build, production dependency audit, Terraform format/validation, Docker build, Trivy source and image scans, and CycloneDX SBOM generation. Scanner gates cover High and Critical findings with available fixes; ignored unfixed findings are not a claim of zero vulnerabilities. Reports are saved for review.

The scanned image is saved as a short-retention Actions artifact. Deployment loads that exact image rather than rebuilding it. Action dependencies use commit hashes, and Dependabot proposes updates. Scan findings should be fixed or reviewed explicitly; do not disable the gate to obtain a green screenshot.

Step 9 Deploy with short lived credentials

After CI succeeds, the production job obtains an AWS role session, logs in to ECR, pushes a unique immutable image tag and reads its digest. The release script validates that digest and writes a versioned Dockerrun bundle. It uploads that bundle over TLS to private S3 and creates an Elastic Beanstalk application version.

`scripts/release_dashboard.py` creates a Standard application version without optional pre-processing (this bundle contains only Dockerrun), updates the named environment and waits for Ready/Green on the requested version. It records the previous version before making the update. Terraform intentionally ignores `version_label` drift so infrastructure ownership and application delivery do not fight each other.

Step 10 Verify the actual release

Acceptance evidence must show all of the following:

1. The Actions verification job succeeded and the deployment job actually ran.
2. OIDC role assumption succeeded without stored AWS access keys.
3. The image digest in the receipt matches the ECR image used by Dockerrun.
4. Beanstalk reports Ready and Green on the expected version.
5. The homepage, built JavaScript asset and `/api/health` return HTTP 200.
6. Health reports that hosted live cloud controls are disabled.
7. Public demo authentication requests are rejected; no passwords are submitted over HTTP.

The release receipt records the commit, run URL, image digest, application version, previous version, endpoint, health and smoke results. A successful Terraform apply alone is not proof that the EBKit application was deployed: the initial Beanstalk sample application may be running before the first release job.

CloudWatch observability

The `infra/terraform/observability.tf` stack defines a CloudWatch dashboard with Beanstalk enhanced health, EC2 CPU, and network traffic. Two alarms monitor unhealthy environment states and sustained CPU above 80 percent. An error query helps find failures in application/platform logs. Beanstalk streams instance logs and retains them for seven days.

The alarms are visible in CloudWatch but have no email or paging subscription configured yet. Missing metrics are treated as missing, not as proof of healthy operation. Memory usage and distributed traces require additional instrumentation; they are not included in this baseline. No separate monitoring server is needed.

Step 11 Demonstrate recovery

For a failed release, inspect Beanstalk Events, enhanced health and CloudWatch logs. Use the receipt's previous application version after review:

```
aws elasticbeanstalk update-environment --region us-east-2 \
  --environment-name ebkit-dashboard-practice --version-label PREVIOUS_VERSION
```

Then repeat the same health and asset smoke checks. Do not claim a rollback test until it was executed and verified. Application rollback does not roll back an external database or destructive schema migration.

Screenshots and evidence to submit

Capture actual tool or browser state, with the run time visible where possible:

1. GitHub branch and commit history showing today's source changes.
2. A successful Actions run with separate verification and deployment jobs.
3. The deploy step showing successful OIDC role assumption, with sensitive identifiers masked where useful.
4. Terraform's reviewed plan and successful apply summary.
5. ECR repository showing the immutable image digest.
6. Beanstalk overview showing the requested version and Ready/Green state.
7. The actual application in a browser and its public-demo notice.
8. The health endpoint and stored smoke-test receipt.
9. A security report and SBOM artifact.

Do not use generated mockups as deployment screenshots. A prior UI design image can be included only when captioned as a design preview. If browser capture is unavailable, provide raw receipts and state which screenshots still need capture. Never screenshot passwords, access tokens, `.env` contents or secret values.

Explain the project to judges

Start with the application problem and show the UI briefly. Then follow one source commit through tests, security checks, the scanned image digest, OIDC login and the running Beanstalk version. Explain why the delivery role cannot administer IAM. Show the deployment receipt and a health check. Finish by explaining the previous version rollback and the cleanup process. Present unfinished HTTPS or protection work as a limitation rather than claiming production readiness.

The root Dockerfile now defaults to public demo mode. While HTTPS is missing, the server rejects authentication and cloud/AI API calls; only health checks and the browser-local simulation remain available. This prevents visitors from spending the operator's AWS or Gemini budget through the public demonstration.

Suggested demonstration order: application idea, source branch, CI results, security report, Terraform plan, OIDC trust, ECR digest, live release, health, recovery and cost controls. This gives a coherent story rather than a list of tools.

Cleanup and production differences

Keep the environment running only while needed for judging. Review Terraform's destroy plan before terminating this named stack. The release bucket is versioned and `force_destroy=false`; retained objects must be reviewed before deletion. ECR images can also block repository deletion. Existing OIDC providers are shared and are not owned or deleted by this stack.

Before production: configure domain and ACM-backed HTTPS, deliberately re-enable authentication, review allowed origins, use a scoped server execution model for customer AWS work, review the already encrypted remote Terraform backend, define service objectives, and choose resilience appropriate to the application. A single instance practice environment is not highly available.

References

The Git and application security implementation covers the native commit hook, Gitleaks history scanning, Semgrep, pip-audit, Checkov, passive ZAP, CODEOWNERS and the owner's requested solo-maintainer branch protections.

- AWS Docker configuration
- AWS enhanced health API configuration
- AWS secret references in environment variables
- AWS Standard application-version processing
- AWS Beanstalk platform bucket permissions
- AWS S3 blocking SSE-C encryption

- GitHub OIDC with AWS
- GitHub deployment environments
- Terraform Elastic Beanstalk environment resource

Verified execution: 17 October 2026

GitHub Actions run 39049823659 completed successfully: **security, verify and deploy all passed**. The deploy job exchanged GitHub OIDC identity for temporary AWS credentials and deployed the already scanned image. No long-lived AWS access keys were stored in GitHub.

- Source commit: `6a0b24369220a7755307ca7390824cad7b611605`.
- Beanstalk version: `release-6a0b24369220-39049823659-2`.
- Environment: `ebkit-dashboard-practice`, Ready / Green.
- ECR image digest: `sha256:be5d0f767b516235d38d9bce5b91aa986232b926f40dfc31d3a0c9b66afd112f`.
- Live URL: http://ebkit-dashboard-practice.eba-cktxxx2w.us-east-2.elasticbeanstalk.com
- Tests: **65 targeted Python tests and 224 frontend tests passed**.
- Trivy source and image reports: zero findings under the configured fixed HIGH/CRITICAL vulnerability and secret gates; SBOM retained.
- Exact Python runtime dependency audit: zero known vulnerabilities. Frontend production dependency audit passed.
- Gitleaks history and local Semgrep rules: zero findings after the documented exact historical fixture exception.
- Checkov: Terraform 41 passed / 0 failed / 7 documented resource-local skips; Docker 131 passed / 0 failed; Actions 148 passed / 0 failed.
- Passive ZAP against the isolated application container: 63 passing checks, zero FAIL findings and four warning categories.

The ZAP warnings concern missing CSP, non-storable content, SPA coverage and cross-origin embedder policy. Passive scanning is limited coverage, especially for browser-driven authenticated flows. A PASS for an HTTP-inapplicable check such as HSTS does not establish HTTPS. These results do not assert zero risk.

Post-deployment smoke tests verified the homepage, built assets and API health. Authentication, deployment, repository and upload API requests were rejected with 403 in public demo mode. The default endpoint remains **HTTP**: login and cloud/AI operations stay gated until a domain and ACM-backed HTTPS are configured.

CloudWatch evidence contains nine application/platform log groups with seven-day retention, a dashboard, enhanced health and two alarms in OK state. Alarm notifications, application traces and memory metrics are follow-up work. Terraform's final full plan reported no changes. The deployed source is the commit above; subsequent documentation-only commits do not replace this release.

Deployment troubleshooting and evidence boundaries

Early image scans blocked vulnerable runtime build tooling. Removing unnecessary package managers and auditing the exact final Python inventory resolved the gate. The immutable repository/environment OIDC subject resolved initial trust failures. Beanstalk subsequently exposed downstream permissions needed for Standard Docker updates: the named platform bucket, this environment's runtime prefix, the application's embedded-extension prefix, launch-template inventory and SuspendProcesses/ResumeProcesses on this environment's Auto Scaling group only. Unrelated applications, secrets and Terraform state remain outside the delivery role's direct access. A deployment maintainer can nevertheless change application code that reads runtime secrets; Secrets Manager does not isolate secrets from trusted maintainers who control deployed code.

An earlier operator-profile diagnostic deployed the identical candidate and proved the image and bundle worked. That was diagnostic evidence only. The final successful GitHub run linked above independently proves OIDC delivery. A previously queued retry of an older commit later replaced the active version; rerunning only the latest commit's OIDC deploy job restored the verified image. Do not retry older release jobs after a newer deployment without deliberately treating that as a rollback. Optional application-version pre-processing is disabled for the Dockerrun-only Standard bundle; six regression cases check that contract and reject failed versions regardless of capitalization. Build, scanning and Ready/Green checks remain mandatory.

GitHub read-back confirmed required pull requests and strict verify/security checks on main, administrator enforcement, disabled force pushes/deletion, secret scanning, push protection and Dependabot security updates. CODEOWNERS uses @Rakesh-Patra. Independent approvals are intentionally not mandatory for the solo-maintainer repository; enable them when another reviewer joins. The native commit hook is installed and its staged-index behavior has seven passing tests.

The seven Terraform skips are explained in DEVSECOPS.md: encryption choices, single-region scope, absent S3 auditing/event consumers, pinned platform maintenance and a confirmed Checkov enhanced-health namespace mismatch. No Docker or Actions exceptions were needed. The read-only filesystem, dropped capabilities and resource limits were exercised on the isolated scan container; they are not automatically applied to live Beanstalk by Dockerrun v1.

The screenshots and release receipt following this guide were captured from the real deployed application, public workflow run and actual passive-scan report. Runtime credentials, secret values and Terraform state are excluded.

EBKit Git and application security

These controls extend the deployment walkthrough. Source/configuration changes here must pass CI before deployment. Repository settings and local hook installation are separate actions; committing their files does not enable them in GitHub or on another developer's laptop.

Keep private material out of Git and Docker

`.gitignore` excludes local environment files, private keys, AWS profiles, Terraform state/variables/plans and generated artifacts. `.dockerignore` excludes the same sensitive inputs from the build context. The root Dockerfile also uses specific COPY paths. Never solve a scan failure by committing a real key as a test input. Use synthetic values assembled inside tests.

Ignored files can still be force-added. The staged-path check rejects private configuration even in that case. It prints paths only, never file contents. Example environment files and Terraform variable examples remain permitted.

Already committed credentials require revocation/rotation. Deleting a file or adding an ignore rule does not invalidate a credential or remove older commits.

Choose one local commit-hook installation

Native Git hook, using Python and Gitleaks 8.30.1 on PATH:

```
git config --local core.hooksPath .githooks
```

The tracked `.githooks/pre-commit` first checks staged paths, then runs Gitleaks against the staged snapshot with redaction. Missing Gitleaks blocks the commit. It does not scan unstaged developer configuration or print secret values.

Alternatively, use the pre-commit framework:

```
python -m pip install pre-commit==4.7.0
git config --local --unset core.hooksPath
pre-commit install
pre-commit run --all-files
```

Choose this alternative only when replacing the native hook configuration. The Gitleaks hook revision is pinned to the reviewed release commit. Review `pre-commit autoupdate` changes before committing them. Local hooks can be bypassed; the full-history CI scan and protected branch are the server-side enforcement.

Secret scanning and historical test data

```
gitleaks git . --redact --config .gitleaks.toml
```

CI checks out complete history, verifies the pinned Gitleaks release archive's SHA256 digest, and scans the repository history. It keeps a redacted JSON report. The configuration extends all upstream default rules.

One reviewed historical sanitizer test used a numeric-only synthetic PAT-shaped input. `.gitleaksignore` excludes only its exact commit/path/rule/line fingerprint. No test directory, credential rule or whole commit is broadly excluded.

GitHub secret scanning and push protection provide another layer when enabled in repository settings. They complement Gitleaks; their provider/rule coverage differs.

Source, dependency and infrastructure gates

- SAST: Semgrep checks reviewed local rules for dynamic execution, shell commands, disabled TLS verification, unsafe YAML and pickle use in application/tool code. Rules stay in the repository, with metrics disabled and no registry ruleset or source upload. This small ruleset is not comprehensive taint analysis.
- SCA: `npm audit --omit=dev --audit-level=high` checks shipped JavaScript packages. `pip-audit --strict` checks the exact third-party Python versions extracted from the built runtime image with standard-library package metadata. The local EBKit distribution is covered by source checks, rather than a public package lookup. Development/scanner dependencies stay outside that runtime inventory. Trivy additionally scans the actual runtime image.
- IaC: Checkov scans Terraform, the root Dockerfile and GitHub Actions. Findings fail the security job. Any accepted exception must be narrow and explain its deployment-specific reason next to the resource; no blanket soft-fail is used.
- Container and source scanning: Trivy gates fixed HIGH/CRITICAL vulnerabilities and secret findings. Its `ignore-unfixed` policy is a coverage limit, not evidence that an unfixed vulnerability is safe. The image SBOM records dependencies.

Pinned scanner dependencies are under `security/`; Dependabot reviews them weekly, alongside app dependencies, the digest-pinned Docker base and Actions. Updating a tool can change findings and must receive the same review as application code.

Passive DAST and runtime isolation

CI runs the already scanned image in a disposable Docker network with demo mode enabled. The app has no AWS credentials, no host port publication, and no mounted developer files. The test uses a read-only root filesystem, writable temporary/job mounts, dropped capabilities, no new privileges, and CPU/memory/PID limits.

A digest-pinned official OWASP ZAP container spiders that local target and performs passive checks. Missing anti-clickjacking or content-type protections fail the gate. Other warnings remain in the JSON/HTML report for review; `-I` permits WARN results but does not permit FAIL findings or scanner errors. This is passive DAST, not an active attack scan or a test of authenticated/cloud workflows.

The server sends anti-framing, nosniff, referrer and permissions headers. CSP is still a reported warning: the application injects runtime configuration inline, so a strict policy needs nonce/hash support and integration testing. HSTS remains pending HTTPS. Do not add HSTS to an HTTP-only demo and claim that HTTPS exists.

The local baseline completed with 63 passing checks, no FAIL findings and four warning categories: missing CSP, non-storable responses, SPA coverage advice, and missing cross-origin embedder policy. These are retained for review. Cache policy and cross-origin isolation need compatibility testing; the SPA warning also means this traditional spider is not complete coverage of the React dashboard.

These Docker runtime flags are exercised in CI. They are not automatically applied to the Elastic Beanstalk host by Dockerrun v1. The live container runs nonroot and uses a multistage build, but the host needs separate platform-specific hardening validation. Distroless is not a drop-in runtime here: the app includes Python CLI tools, Git and Node, rather than a Node-only static service.

Branch protection, ownership and RBAC

`security/main-protection.json` is the reviewable desired policy for `main`: required PR, current `verify` and `security` checks, dismissal of stale approvals when reviews are used, resolved conversations, linear history, and no force pushes or branch deletion. Administrators are included in enforcement. At the owner's request this is a solo-maintainer policy: the required independent approval count is zero, and code-owner/last-push approval are not mandatory. When collaborators join, set the count to at least one and enable code-owner and latest-push approval.

```
python scripts/harden_github.py          # preview
python scripts/harden_github.py --apply  # change settings and save a read-back receipt
```

`.github/CODEOWNERS` initially names the real repository owner, including sensitive CI, Terraform, Docker, auth and security paths. Code-owner rules must exist on the PR's base branch to take effect. An author cannot approve their own PR; add a trusted independent reviewer before expecting an owner-authored PR to merge.

Give administrators settings access only when needed, maintainers merge/release responsibility, developers PR access, and auditors read access. Teams/custom roles depend on the account plan; do not pretend a personal repository has organization teams. Require MFA for maintainers, remove stale collaborators, review installed GitHub Apps, and keep production environment/deployment branch restrictions.

Actions start with `contents: read`. Only the protected deployment job gets `id-token: write`; PRs never receive AWS credentials. Deploy waits for both security and verification jobs and publishes the same scanned image by immutable digest.

AI scanner integration from the supplied example

The pasted Sim.ai workflow is not enabled. First confirm ownership of that workflow, what repository/PR data it retains, and who can access the result. Keep its API key in a protected CI secret store and never interpolate PR titles/bodies directly into shell source. Build a JSON payload from `GITHUB_EVENT_PATH` with a JSON parser. Use bounded timeouts, `curl --fail-with-body`, and validate the returned scan status; a successful HTTP trigger is not a completed security assessment. Fork PRs must not receive integration credentials. A reviewed GitHub App/webhook can avoid executing third-party integration credentials in untrusted PR code.

Evidence and remaining work

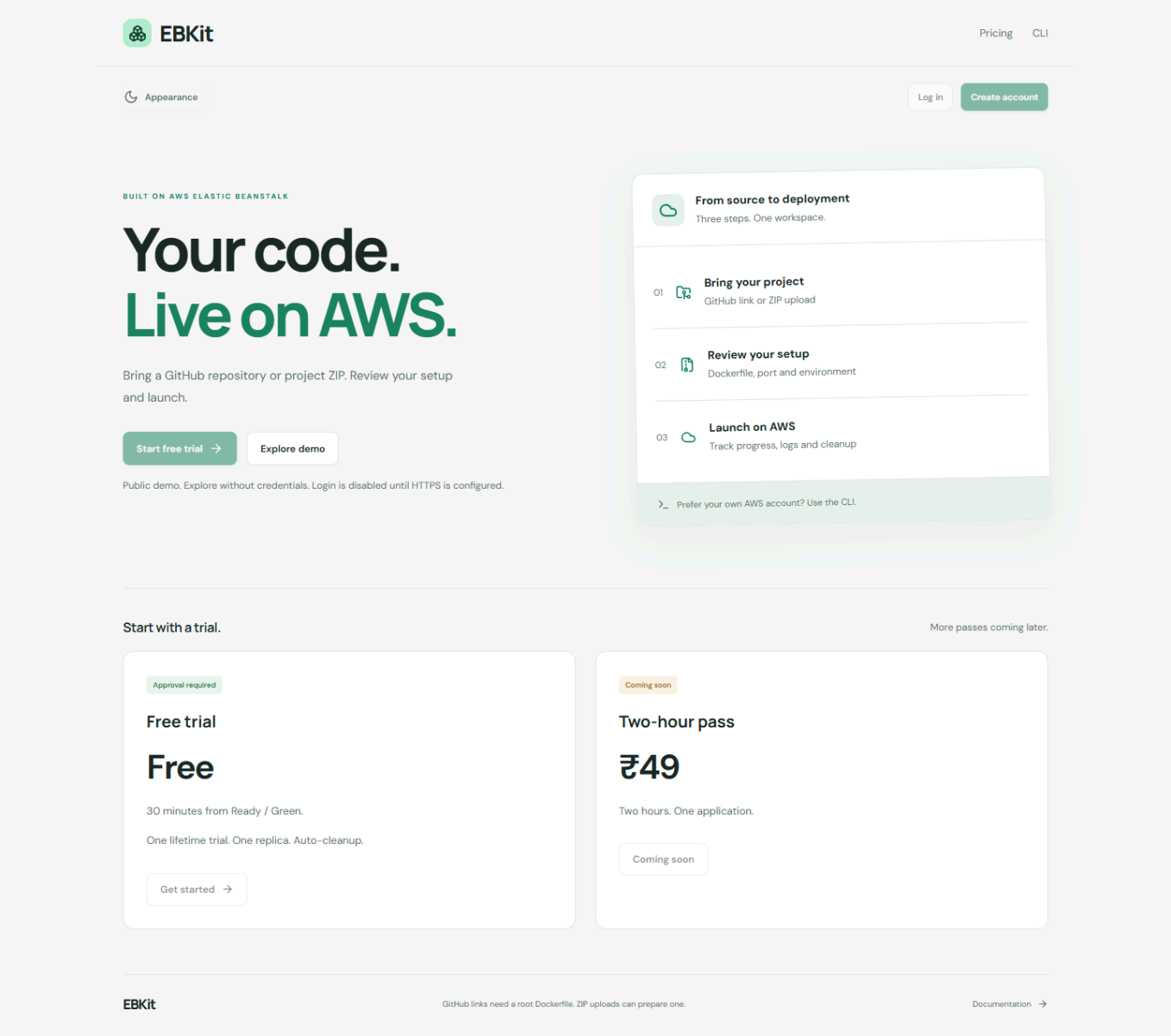
Keep redacted Gitleaks, Semgrep, Checkov, dependency, Trivy, ZAP and SBOM artifacts from the same source commit. Read the actual job conclusions before claiming a gate passed. The walkthrough records live deployment evidence separately from prepared configuration. HTTPS, a reviewed CSP, active/authenticated DAST, alarm notifications, and independent review remain distinct follow-up work until verified.

References: Gitleaks, Semgrep CLI, Checkov CLI, ZAP baseline, GitHub branch protection.

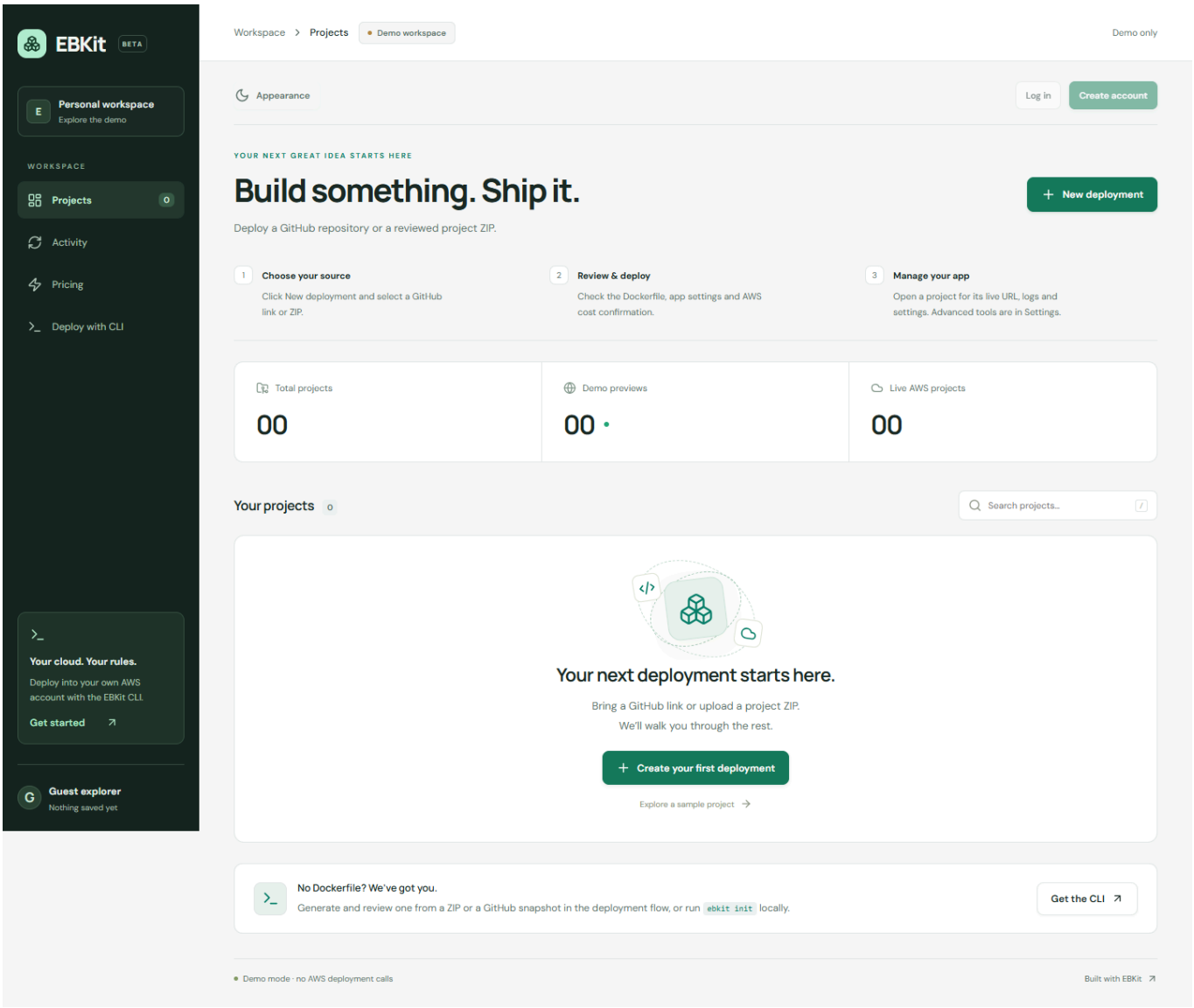
Verified release evidence

Capture time 2026-10-17T16:54:00.684653+00:00

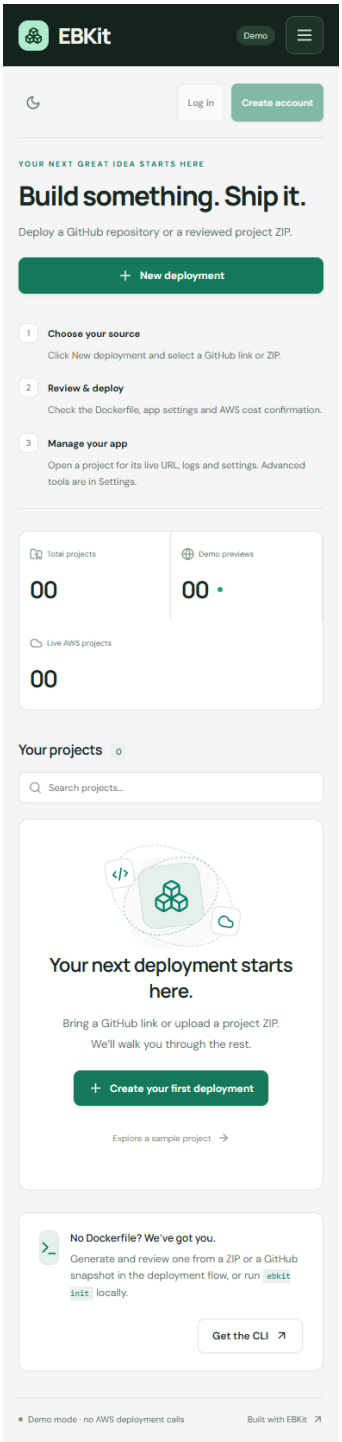

```
{
  "commit": "6a0b24369220a7755307ca7390824cad7b611605",
  "runUrl": "https://github.com/Rakesh-Patra/ebready/actions/runs/39049823659",
  "verified": true,
  "homepageStatus": 200,
  "healthStatus": 200,
  "assetStatus": 200,
  "publicDemoGuards": {
    "/insforge/api/auth/sessions": 403,
    "/api/deploy": 403,
    "/api/repository": 403,
    "/api/uploads": 403
  },
  "verifiedAt": "2026-10-17T16:51:23.856945+00:00",
  "url": "http://ebkit-dashboard-practice.eba-cktxxx2w.us-east-2.elasticbeanstalk.com",
  "awsStatus": "Ready",
  "awsHealth": "Green",
  "httpsConfigured": false,
  "imageDigest": "sha256:be5d0f767b516235d38d9bce5b91aa986232b926f40dfc31d3a0c9b66afd112f"
}
```



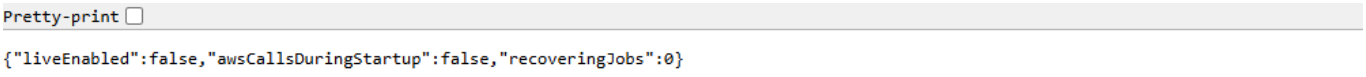
Deployed application landing page



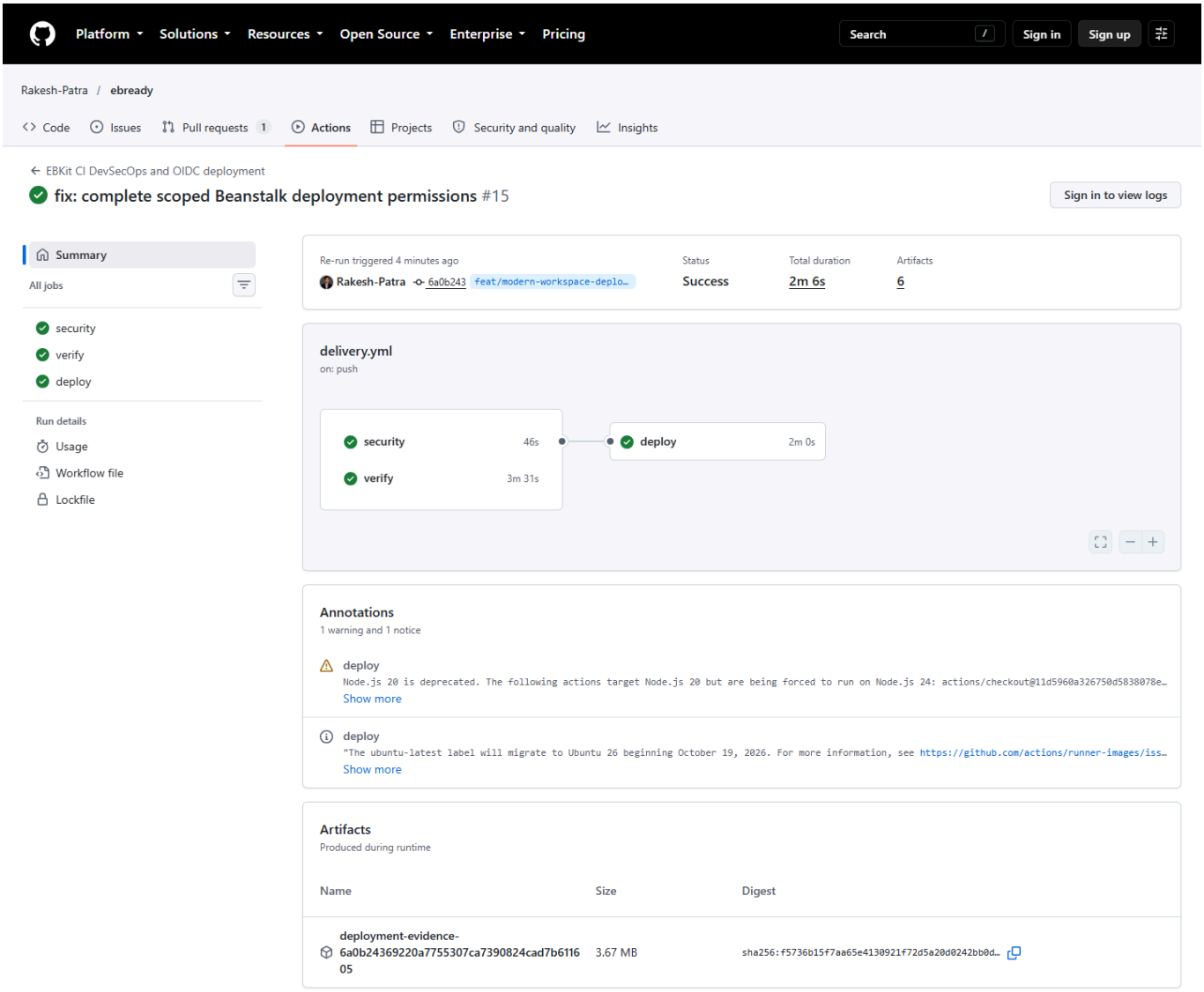
Guest dashboard served by the verified AWS release



Deployed dashboard at phone width



Health response from the deployed API



Actual public GitHub Actions run; see the linked run for individual steps

ZAP Scanning Report

Site: http://ebkit-dast:8080

Generated on Sat, 17 Oct 2026 16:36:48

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	3
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	5
Info	Informational	http://ebkit-dast:8080	Percentage of responses with status code 2xx	66 %
Info	Informational	http://ebkit-dast:8080	Percentage of responses with status code 4xx	33 %
Info	Informational	http://ebkit-dast:8080	Percentage of endpoints with content type text/css	25 %
Info	Informational	http://ebkit-dast:8080	Percentage of endpoints with content type text/javascript	25 %
Info	Informational	http://ebkit-dast:8080	Percentage of endpoints with content type text/plain	50 %
Info	Informational	http://ebkit-dast:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://ebkit-dast:8080	Count of total endpoints	4

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	1

Actual ZAP report for the isolated CI container; warnings require review